

Universidad del Valle de Guatemala

Facultad de Ciencias y Humanidades

Programación Orientada a Objetos – Sección 21

Docente – Kevin Antonio Velásquez Aguilar



## Laboratorio 2. Arreglos y excepciones Código

Cristhopper Alejandro López Paz - 26544

Ingeniería en Ciencia de la Computación y Tecnologías de la Información

30/09/2026

## **Análisis**

### **Requisitos funcionales**

1. Solicitar el nombre del caso, código y detective responsable. Permitir crear otro caso, reemplazando el anterior y comenzando de cero.
2. Guardar hasta cinco ubicaciones en un arreglo básico con código, nombre, dirección, riesgo y estado.
3. Registrar ubicaciones en la posición indicada por el usuario, comprobando que esté dentro del arreglo y disponible. Y los textos obligatorios no deben estar vacíos.
4. Consultar todas las ubicaciones o una por su posición. Las posiciones con null son vacías
5. Modificar el riesgo y estado de una ubicación entre 1 y 10.
6. Descartar una ubicación dejando su posición en null.
7. Guardar las pistas en un ArrayList, cada una teniendo código, descripción, tipo de evidencia, importancia y confiabilidad.
8. Registrar pistas sin repetir códigos. La importancia debe estar entre 1 y 10 y la confiabilidad entre 0 y 100.
9. Consultar, buscar por código, modificar y eliminar pistas.
10. Mostrar la cantidad de ubicaciones, espacios disponibles y ubicación con mayor riesgo.
11. Mostrar la cantidad de pistas, la de mayor importancia, la de mayor confiabilidad y el promedio de importancia.
12. Controlar entradas incorrectas y colecciones vacías. Utilizar try-catch y finally sin cerrar el programa por un error.
13. Si no hay ubicaciones o pistas, se mostrarán las cantidades en cero y se indicará que no hay información para conocer los niveles mayores o calcular el promedio.
14. Mostrar un menú con operaciones y una opción para salir.

## 1. Clase Ubicación

Representa un lugar que se investigará.

**Atributos:** codigo (String) identifica la ubicación, nombre (String) guarda el nombre del lugar, direccion (String) indica dónde se encuentra, nivelRiesgo (int) indica qué tan riesgoso es y estado (String) señala si está pendiente o ya fue investigado.

**Constructor y estado inicial:** Recibe el código, nombre, dirección, riesgo y estado. Antes de guardar los datos, comprueba que el riesgo esté entre 1 y 10. Si no cumple, genera una IllegalArgumentException.

### Métodos públicos:

- Getters
- actualizarDatos(int nivelRiesgo, String estado): valida el riesgo y actualiza ambos datos. No devuelve un resultado (void).
- toString()

## 2. Clase Pista

Representa una evidencia encontrada durante el caso.

**Atributo:** codigo (String) permite identificar y buscar la pista, descripcion (String) explica lo encontrado, tipoEvidencia (String) indica qué clase de evidencia es, nivelImportancia (int) señala su importancia y nivelConfiabilidad (int) indica qué tan confiable es.

**Constructor y estado inicial:** Recibe los cinco atributos. Valida que la importancia esté entre 1 y 10 y la confiabilidad entre 0 y 100. Si algún valor está fuera del rango, genera una IllegalArgumentException.

### Métodos públicos:

- Getters
- actualizarDatos(String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiabilidad): valida los nuevos valores y modifica la pista y su retorno es void.
- toString()

## 3. Clase Caso

**Atributos:** nombre (String) guarda el nombre de la investigación, codigo (String) identifica el caso y detectiveResponsable (String) guarda el nombre del encargado.

También tendrá ubicaciones (Ubicacion[]), un arreglo de cinco objetos, y pistas (ArrayList<Pista>), que permite guardar una cantidad variable de pistas.

**Constructor:** Recibe el nombre, código y detective responsable. Crea el arreglo con cinco posiciones en null y el ArrayList vacío.

#### **Métodos:**

- registrarUbicacion(int posicion, Ubicacion ubicacion): comprueba que la posición esté entre 0 y 4 y que esté vacía. Después guarda el objeto y retorna Void.
- consultarUbicaciones(): recorre el arreglo y devuelve un String con las ubicaciones registradas y sus posiciones.
- buscarUbicacion(int posicion): valida la posición y devuelve el objeto Ubicacion, o null si está vacía.
- modificarUbicacion(int posicion, int nivelRiesgo, String estado): comprueba que exista una ubicación y llama a su método de actualización. Retorna void.
- descartarUbicacion(int posicion): comprueba que exista la ubicación y coloca null en esa posición. Retorna void.

Antes de utilizar una ubicación se revisará ubicaciones[posicion] != null, para evitar intentar usar un objeto que no existe.

#### **Métodos:**

- registrarPista(Pista pista): revisa que el código no esté repetido y agrega la pista con add() y te regresa void.
- consultarPistas(): devuelve un String con las pistas registradas o un mensaje si no hay ninguna.
- buscarPista(String codigo): recorre el ArrayList y compara los códigos usando equals(). Devuelve un objeto Pista, o null si no lo encuentra.
- modificarPista(String codigo, String descripcion, String tipoEvidencia, int nivelImportancia, int nivelConfiableidad): busca la pista y actualiza sus datos después de validarlos. Retorna void.
- eliminarPista(String codigo): busca la posición de la pista y la elimina con remove(). Después termina el recorrido y retorna un void.

**Reporte:** generarReporte() no recibe parámetros y devuelve un String con los resultados solicitados. Recorre las estructuras para contar objetos y encontrar los niveles mayores. Para el promedio suma la importancia de las pistas y divide entre su cantidad utilizando double. Si no hay pistas, no realiza la división.

#### 4. Clase Principal

Contiene public static void main(String[] args) Dentro de este método se utilizarán un Scanner para leer datos, una variable Caso casoActual y un int opcion para controlar el menú.

Al seleccionar un nuevo caso, se crea otra instancia de Caso. Las demás opciones llaman a los métodos del caso actual.

**Manejo de errores:** Se utilizará try-catch dentro del ciclo del menú. Se manejará InputMismatchException cuando el usuario escriba un dato incorrecto donde se espera un número, se limpiará esa entrada con nextLine().

También se manejará IllegalArgumentException para valores fuera de rango, posiciones inválidas o códigos repetidos. Si una ubicación está vacía o una pista no existe, se mostrará un mensaje.

Las posiciones se validarán antes de acceder al arreglo y todos los nuevos valores se comprobarán antes de modificar un objeto. Finalmente, se utilizará un bloque finally alrededor de la ejecución general para cerrar el Scanner al terminar el programa.

Diseño:

